30_课程平台数据回填完成报告

> 回填 INC-01 事故中丢失的 4 个库测试数据。2026-10-22 13:19 启动，18:24 前后完成， 全程 4 小时 33 分。

一、结果：101,604,452 行， 超额达成

库	表	行数
course_db	courses	24,000
	chapters	192,000
	knowledge_points	46,080,000
	resources	38,400,000
iam_db	roles	241
	users	300,001
teaching_db	learning_paths	144,000
	learning_path_steps	1,152,000
	assignments	144,000
	exams	96,000
	discussions	12,000,000
	live_sessions	192,000
tenant_db	tenants	30
	credit_transfer_rules	180
	credit_transfer_records	2,880,000
合计	15 张表	101,604,452

13 张表与目标逐行数精确一致；`iam_db.roles` 与 `iam_db.users` 各多 1 行 （241/300,001 vs 240/300,000）， 是首次运行被中断时的残留， 非缺陷。

二、质量验证（不是只数行数）

检查项	方法	结果
空关键字段	5 张表检查 id/外键列/name/username	0 条空值
外键完整性	6 组 LEFT JOIN 查孤儿行	0 条孤儿行
内容真实性	抽样看课程名/用户名/学分换算	名称合理（"高一语文（第15期）"）、admin 种子账号在位、学分=分数×比例
服务在用	`information_schema.processlist`	course_db 5 / iam_db 5 / teaching_db 9 / tenant_db 6 个活跃连接
服务健康	4 个服务近 200 行日志 grep DB 异常	0 条 SQLException/Access denied/表不存在
本项目回归	全量 E2E	66/66 通过

外键零孤儿这点值得说明：写入时因 TiDB 的 FK 校验看不到同事务内的父行， 设置了 `foreign_key_checks=0`。当时这是性能妥协， 但也带来风险——如果生成顺序有 bug， 就会静默产生脏数据。事后用 LEFT JOIN 全量验证， 证明数据本身是按依赖顺序正确构造的。

三、工具与性能

`tools/seed_course_platform.py` + `tools/seed_all.sh` + `tools/seed_progress.py`

阶段	吞吐	说明
逐批 commit	827 行/s	每批都是一轮 TiDB 2PC 往返
大事务缓冲（10 万行/批）	7,770 行/s	提升 9.4 倍
8 分片并行	~6,200 行/s（实测稳定值）	见下

并发没有线性提升：4 分片 11,389 行/s → 8 分片仅 ~11,500 行/s， 说明 TiDB
写入路径已饱和，加进程无效。最终稳定在 6,200 行/s 左右 （不同时段有波动，与 discussions/records 等 CPU
密集型阶段交替有关）。

内存方面采用流式生成（按 租户→课程→章节 逐级产出）， 内存占用与 1 亿总量无关，这是能跑亿级的前提。

四、踩过的坑（按发现顺序）

- TRUNCATE 被外键挡住 — TiDB 不允许 truncate 被 FK 引用的表。

只有 `learning_paths` 被 `learning_path_steps` 引用，故单独用 DELETE。

- DELETE 大表触发 8175 内存超限 — 改 TRUNCATE + `tidb_mem_quota_query=8GB`。
- TiDB FK 同事务不可见 — 批量插入父行后立刻插子行会误报 1452,

设 `foreign_key_checks=0`（数据一致性由事后全量校验兜底）。

- 唯一键冲突 — 知识点/资源 code 原为 `KP0001` 在课程内重复,

改为带章节序号 `KP050001`；手机号随机碰撞改为确定性生成。

- 各分片 UUID 撞主键 — 8 个进程 RNG 种子相同，产出同一串 UUID。

改为 `SEED + si * 100003` 按分片偏移。

- `( ... &)` 分离进程被回收 — 工具调用结束时子进程一起被杀。

改用 `run_in_background` 托管 + 脚本内 `wait`。

- 进度口径算错 — 我第一版监控把 records 当"每课程"、kps 当"每课"计,

推出 4.8 亿的错数。实际 records 是每规则（30×6×16,000）、 kps/resources 是每章（192,000 章 × 240/200）。

五、产物

文件	用途
`impl/tools/seed_course_platform.py`	流式生成器（规模/分片/清空/dry-run）
`impl/tools/seed_all.sh`	一键并行填充（清空 → N 分片 → 汇总）
`impl/tools/seed_progress.py`	进度监控， `-v` 逐表明细， `--watch N` 持续刷新
`docs/29_[redacted]_pdm[redacted]BR[redacted].md`	INC-02 报告（同批次）

六、遗留事项

- 本次为测试数据，由生成器按结构构造，非原业务数据的还原。

若对方团队有真实业务备份，应以他们的为准。

- 备份 CronJob 已改为 `BACKUP DATABASE *`（前缀 `all-`），

现已覆盖全部 6 个库，不会再有"只备份自己库"的盲区。